UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report – 04

COURSE CODE: CSE 402

COURSE TITLE: Operating System Lab

SUBMITTED BY:

NAME	: Swagoto Utsab Singha Roy
STUDENT ID	: 23101124
SEMESTER	: 4th YEAR 1st SEMESTER
SECTION	: C – 1
DATE	: 04 – 09 – 2026

SUBMITTED TO:

ATIA RAHMAN ORTHI
LECTURER
DEPARTMENT OF CSE,UAP

1. Problem Statement

Implement the Shortest Job First (SJF) Scheduling Algorithm in both Non-Preemptive and Preemptive (SRTF) variants, and compare them with the First Come First Serve (FCFS) CPU Scheduling Algorithm for the following processes.

Table 1: Processes used for SJF (Non-Preemptive and Preemptive)

Process	Arrival Time (AT)	Burst Time (BT)
P1	0	4
P2	1	2
P3	3	2
P4	2	1

Table 2: Processes used for FCFS

Process	Arrival Time (AT)	Burst Time (BT)
P1	3	5
P2	2	4
P3	4	3
P4	1	2
P5	5	3

For all three algorithms, calculate:

- Completion Time (CT)
- Turnaround Time (TAT)
- Waiting Time (WT)
- Average Turnaround Time (AVG TAT)
- Average Waiting Time (AVG WT)

Also compare the Average TAT and Average WT across all three algorithms to determine which scheduling approach performs best for the given processes.

2. Source Code

```
# sjf
process = ["P1", "P2", "P3", "P4"]
at = [0, 1, 3, 2]
bt = [4, 2, 2, 1]

n = len(process)

ct = [0] * n
wt = [0] * n
tat = [0] * n
done = [0] * n

time = 0
completed = 0

```

```
# SJF Non-Preemptive
while completed < n:
    x = -1

    for i in range(n):
        if at[i] <= time and done[i] == 0:
            if x == -1 or bt[i] < bt[x]:
                x = i

    if x == -1:
        time += 1
    else:
        time += bt[x]
        ct[x] = time
        done[x] = 1
        completed += 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]

sjf_avg_tat = sum(tat) / n
sjf_avg_wt = sum(wt) / n

def print_table(title, headers, rows):
    col_w = 10
    print(f"\n {title} ")
    print("".join(f"{h:^{col_w}}" for h in headers))
    for row in rows:
        print("".join(f"{str(v):^{col_w}}" for v in row))

sjf_rows = [
    [process[i], at[i], bt[i], ct[i], tat[i], wt[i]]
    for i in range(n)
]

print_table(
    "SJF - Non-Preemptive",
    ["Process", "AT", "BT", "CT", "TAT", "WT"],
    sjf_rows
)

print(f"\nAverage TAT = {sjf_avg_tat:.2f}")
print(f"Average WT = {sjf_avg_wt:.2f}")

# SJF Preemptive (SRTF)
remaining = bt.copy()

ct_pre = [0] * n
tat_pre = [0] * n
wt_pre = [0] * n

time = 0
completed = 0

while completed < n:
    x = -1

```

```
    for i in range(n):
        if at[i] <= time and remaining[i] > 0:
            if x == -1 or remaining[i] < remaining[x]:
                x = i

    if x == -1:
        time += 1
    else:
        remaining[x] -= 1
        time += 1

        if remaining[x] == 0:
            ct_pre[x] = time
            completed += 1

for i in range(n):
    tat_pre[i] = ct_pre[i] - at[i]
    wt_pre[i] = tat_pre[i] - bt[i]

pre_avg_tat = sum(tat_pre) / n
pre_avg_wt = sum(wt_pre) / n

pre_rows = [
    [process[i], at[i], bt[i], ct_pre[i], tat_pre[i], wt_pre[i]]
    for i in range(n)
]

print_table(
    "SJF - Preemptive SJF",
    ["Process", "AT", "BT", "CT", "TAT", "WT"],
    pre_rows
)

print(f"\nAverage TAT = {pre_avg_tat:.2f}")
print(f"Average WT = {pre_avg_wt:.2f}")

# FCFS
lst = [
    ["P1", 3, 5],
    ["P2", 2, 4],
    ["P3", 4, 3],
    ["P4", 1, 2],
    ["P5", 5, 3]
]

lst.sort(key=lambda x: x[1])

ct2 = []
tat2 = []
wt2 = []
time = 0

for name, arr, burst in lst:
    if time < arr:
        time = arr
    time += burst
    ct2.append(time)
    turnaround = time - arr
```

```
    tat2.append(turnaround)
    wt2.append(turnaround - burst)

avg_tat = sum(tat2) / len(tat2)
avg_wt = sum(wt2) / len(wt2)

fcfs_rows = [
    [lst[i][0], lst[i][1], lst[i][2], ct2[i], tat2[i], wt2[i]]
    for i in range(len(lst))
]

print_table(
    "FCFS",
    ["P_id", "AT", "BT", "CT", "TAT", "WT"],
    fcfs_rows
)

print(f"\nAverage TAT = {avg_tat:.2f}")
print(f"Average WT = {avg_wt:.2f}")

# Comparison
print("\n\nCOMPARISON")

print(
    "Non-Preemptive SJF is better in terms of TAT than FCFS"
    if sjf_avg_tat < avg_tat
    else
    "FCFS is better in terms of TAT than Non-Preemptive SJF"
)

print(
    "Non-Preemptive SJF is better WT than FCFS"
    if sjf_avg_wt < avg_wt
    else
    "FCFS is better WT than Non-Preemptive SJF"
)

print(
    "Preemptive SJF is better in terms of TAT than Non-Preemptive SJF"
    if pre_avg_tat < sjf_avg_tat
    else
    "Non-Preemptive SJF is better in terms of TAT than Preemptive SJF (SRTF)"
)

print(
    "Preemptive SJF is better WT than Non-Preemptive SJF"
    if pre_avg_wt < sjf_avg_wt
    else
    "Non-Preemptive SJF is better WT than Preemptive SJF"
)
```

Listing 1: SJF Non-Preemptive, Preemptive (SRTF), FCFS and Comparison

3. Output

SJF – Non-Preemptive

Average TAT = 4.75 Average WT = 2.50

Process	AT	BT	CT	TAT	WT
P1	0	4	4	4	0
P2	1	2	7	6	4
P3	3	2	9	6	4
P4	2	1	5	3	2

SJF – Preemptive (SRTF)

Process	AT	BT	CT	TAT	WT
P1	0	4	9	9	5
P2	1	2	3	2	0
P3	3	2	6	3	1
P4	2	1	4	2	1

Average TAT = 4.00 Average WT = 1.75

FCFS

P_id	AT	BT	CT	TAT	WT
P4	1	2	3	2	0
P2	2	4	7	5	1
P1	3	5	12	9	4
P3	4	3	15	11	8
P5	5	3	18	13	10

Average TAT = 8.00 Average WT = 4.60

Comparison

```
COMPARISON
Non-Preemptive SJF is better in terms of TAT than FCFS
Non-Preemptive SJF is better WT than FCFS
Preemptive SJF is better in terms of TAT than Non-Preemptive SJF
Preemptive SJF is better WT than Non-Preemptive SJF
```

4. Discussion

In this experiment, three CPU scheduling algorithms were implemented and compared: Shortest Job First Non-Preemptive (SJF), Preemptive SJF also known as Shortest Remaining Time First (SRTF), and First Come First Serve (FCFS).

The SJF and Preemptive sections of the code used an identical process set of four processes (P1–P4) with the same arrival and burst times, which makes the comparison between these two variants directly meaningful. The FCFS section used a separately defined five-process set to provide an independent benchmark from a different scheduling philosophy.

For SJF Non-Preemptive once a process starts executing it runs to completion. At time 0, only P1 has arrived, so it starts immediately and finishes at time 4 (CT = 4). By time

4, all remaining processes have arrived; among them P4 has the shortest burst time of 1, so it runs next (CT = 5), followed by P2 (BT = 2, CT = 7) and then P3 (BT = 2, CT = 9). This yields an Average TAT of 4.75 and an Average WT of 2.50.

For SJF Preemptive (SRFP), the scheduler re-evaluates at every clock tick and can interrupt a running process if a newly arrived process has a shorter remaining burst time. At time 0, P1 (BT = 4) starts. At time 1, P2 arrives with BT = 2, which is less than P1's remaining 3, so P1 is preempted. P2 runs until time 3 (CT = 3). At time 2, P4 had arrived with BT = 1, but P2 was already shorter. At time 3, P4 (remaining = 1) is the shortest and finishes at time 4 (CT = 4). P3 arrives at time 3 with BT = 2, and runs next (CT = 6). Finally P1 resumes and finishes at time 9 (CT = 9). This yields a lower Average TAT of 4.00 and Average WT of 1.75, confirming that preemption benefits shorter processes at the cost of delaying longer ones.

For FCFS, processes are served in arrival order. P4 (AT = 1) is first, completing at time 3. P2 (AT = 2) finishes at time 7. P1 (AT = 3, BT = 5) is a long job that delays the remaining processes, producing the convoy effect. P3 and P5, both with BT = 3, must wait behind P1 and accumulate significant waiting time (WT = 8 and 10 respectively). This results in a high Average TAT of 8.00 and Average WT of 4.60.

The comparison clearly shows:SJFP < SJF Non-Preemptive < FCFS in terms of both average turnaround time and average waiting time.SJFP achieves the best performance by aggressively prioritising the shortest remaining job at every moment, but introduces context-switching overhead. SJF Non-Preemptive avoids this overhead while still performing significantly better than FCFS. FCFS, though fair and free of starvation, suffers heavily from the convoy effect whenever a long job arrives early.

It is also worth noting that in SJF (both variants), a process with a very long burst time risks starvation if shorter processes keep arriving. FCFS never starves any process since it serves strictly in arrival order, but this fairness comes at the price of a higher average waiting time.

5. Conclusion

In this experiment, three CPU scheduling algorithms were implemented and compared. For the given process sets, Preemptive SJF (SRTF) achieved the best performance with an Average TAT of 4.00 and Average WT of 1.75, followed by Non-Preemptive SJF (Average TAT = 4.75, Average WT = 2.50), and finally FCFS (Average TAT = 8.00, Average WT = 4.60). These results confirm that ordering jobs by remaining burst time rather than arrival time significantly reduces average waiting time by ensuring that shorter jobs are not blocked behind longer ones. However, SJF variants achieve this improvement at the theoretical cost of possible starvation for long-burst processes, whereas FCFS guarantees every process runs in arrival order with no risk of starvation, at the expense of higher average waiting time. In practice, the choice between these algorithms reflects a trade-off between minimising average waiting time (SJF /SJFP) and guaranteeing fairness and freedom from starvation (FCFS).

6. GitHub

Click here to view source code